MealDash

Business Requirements Document — How I Scoped This Project

The company, and why I set it up this way

MealDash is a fictional quick-commerce food delivery company I built this project around, modeled on how platforms like Zomato and Swiggy actually operate connecting customers, restaurants, and delivery agents across multiple cities. I used a real, public delivery-operations dataset rather than inventing numbers, so the findings would be grounded in real patterns instead of a made-up story.

The problem I set out to solve

The starting point was simple: delivery times vary a lot, and nobody knows exactly why. Some orders take 10 minutes, others take almost an hour. Before I touched any data, I framed this as a real ops pain point today, if someone wanted to know why a particular city or week was slow, they'd have to manually dig through data for a day or two. There's no quick way to check. That manual-investigation gap is really what I was trying to fix: not just ‘find interesting patterns,’ but build something that answers ‘why was it slow’ in minutes, not days.

What I actually wanted to find out

I didn't want to just build a dashboard and hope something useful showed up in it. So before writing any SQL, I wrote down the specific questions I needed answered, in an order that would build understanding step by step starting with how bad the problem is overall, then breaking it into things ops can control (traffic routing, staffing, order bundling) versus things they can't (weather), and finishing with whether things are getting better or worse over time:

1. What is the average and typical (median) delivery time overall?
2. Which city has the slowest average delivery time?
3. Does road traffic density affect delivery time?
4. Does weather condition affect delivery time?
5. Does delivery distance affect delivery time?
6. Do certain vehicle types deliver faster than others?
7. Do higher-rated delivery agents deliver faster?
8. Does handling multiple deliveries in one trip slow delivery time down?
9. Are festival days slower than normal days?
10. Is delivery time trending better or worse over time?

What I planned to deliver

I wanted this to look like something a real analytics team would actually hand over, not just a notebook of queries. So I scoped these deliverables upfront:

What	Why it matters
Cleaned dataset	Nothing downstream is trustworthy if the input data has silent nulls or bad formats
10 SQL queries with reasoning	Answers each question, but also shows how I checked my own numbers before trusting them
Fabric Lakehouse + Dataflow	Makes the pipeline repeatable, not a one-time manual pull
Live Power BI dashboard	Something stakeholders can actually open and explore themselves
Findings write-up	Translates numbers into plain-language takeaways
Recommendations	The actual point numbers alone don't help ops unless they lead somewhere

Tools, and why I picked each one

Tool	Why I used it here
Excel (Power Query)	First cleaning pass — fixing nulls, formats, and building the Distance_km column before anything else touched the data
SQL (Fabric SQL Analytics Endpoint)	Where the actual business questions get answered — aggregations, CASE buckets, and window functions for the trend question
Microsoft Fabric (Lakehouse + Dataflow Gen2)	Keeps the data pipeline repeatable and puts everything under one governed workspace, instead of a one-off manual upload
Power BI	Turns the SQL findings into something a non-technical stakeholder can explore without needing to read a query

How I'd know if this actually worked

I set myself three bars to clear, and I didn't want to call the project done until all three were true:

- Every one of the 10 questions is answered with a query I can explain out loud, not just a result I can show.
- Someone who's never seen this data could open the dashboard and understand the top 2-3 findings within seconds, without me explaining anything.
- Every recommendation is specific enough that ops could act on it this week not vague statements like ‘monitor delivery times more closely.’

What's in scope, and what I deliberately left out

In scope: everything about delivery time itself and what drives it — traffic, weather, distance, vehicle type, agent performance, order bundling, and festival effects.

Deliberately out of scope: pricing, restaurant menu data, and the customer app experience. Not because they don't matter, but because pulling them in would have diluted the analysis — I wanted to answer the delivery-time question thoroughly rather than touch five different problems shallowly.

Assumptions I had to make, and why

- This is a historical, static dataset, not a live feed — so this is a diagnostic project, not a real-time monitoring one.
- The dataset doesn't include a 'promised delivery time,' so I couldn't measure a formal SLA breach rate. Instead I benchmarked against segment averages, which is a fair substitute but worth being upfront about.
- Distance is straight-line (Haversine formula from GPS coordinates), not actual road-route distance — I calculated this myself since the dataset only gave raw coordinates, and I've noted this limitation wherever distance comes up in the findings.
- The recommendations are directional, based on this dataset — I'd want to validate the festival-day effect specifically against a fuller year of data before committing budget to it.

Who this is for

Who	What they'd want from this
Operations Leadership	A way to see where and why delays happen without waiting on a manual data pull
City / Regional Managers	City-level detail they can act on locally
Delivery Agent Management	Visibility into what separates fast agents from slow ones, for training and routing decisions

Rough plan I followed

Phase	What I did
1	Cleaned the raw data in Excel Power Query — nulls, formats, categories
2	Answered all 10 business questions in SQL against the Fabric Lakehouse
3	Set up the Fabric Lakehouse and Dataflow Gen2 pipeline
4	Built the Power BI dashboard on top of the same data
5	Wrote up findings and turned them into specific recommendations